

Один глобальный луч на множестве GPU:

Точный высокопроизводительный beam search с frontier почти в миллиард состояний

Анонимные авторы

Аннотация

Мы представляем первую опубликованную высокопроизводительную multi-GPU систему beam search, которая физически шардирует один логический frontier между устройствами и совместно строит один следующий луч. Генерация кандидатов, оценка, дедупликация по 128-битному ключу, маршрутизация и хранение frontier остаются на GPU; на host поступают только компактные управляющие данные и метаданные родословной. При отсутствии семантических локальных ограничений шардов потоковая процедура возвращает тот же глобальный top- B по редуцированной оценке ключа, что и монолитная реализация, при той же квантизации оценки, порядке разрешения равных оценок для фиксированной раскладки и заявленной 128-битной модели идентичности. На восьми A100 40GB результат по ёмкости фиксирует запрос удерживаемого frontier $B_{\text{req}} = 770,883,178$; выравнивание даёт $B_{\text{eff}} \geq B_{\text{req}}$ и, следовательно, не менее 18,501,196,272 номинальных потомков до дедупликации на насыщенную глубину при 24 генераторах. Этот результат не содержит wall time или точного B_{eff} и используется только как доказательство ёмкости. Отдельно измеренный запуск на двух T4 при $B_{\text{eff}} = 82,837,504$ имеет 1,988,100,096 номинальных потомков на насыщенную глубину; глубины 7–10 занимали в среднем 65.670 с, то есть 30.274 миллиона логических потомков/с end to end. Этот результат по пропускной способности относится к двум GPU и не устанавливает strong scaling за их пределами. Новизна состоит в полной системной совокупности свойств, а не в параллельном, GPU- или распределённом beam search по отдельности.

1 Введение и постановка задачи

Beam search становится задачей распределённых систем, когда удерживаемый frontier перестаёт помещаться в память одной GPU. Расширение, оценка, дедупликация, глобальный отбор и родословная должны распределяться без превращения одного поиска в независимые реплики и без переноса frontier через host memory. Мы исследуем, может ли GPU-кластер сохранить семантику одного монолитного луча, масштабируя его ёмкость суммарной памятью устройств.

Пусть $G = (V, E)$ — неявный ориентированный граф, $A_a(x)$ — поток состояний x под действием генератора $a \in \{1, \dots, m\}$, а $\sigma(x, a)$ — детерминированная вещественная функция оценки. Математический контракт и стримы 2–5 допускают обученную или заданную вручную функцию, если стрим 1 выдаёт детерминированные квантованные ключи; поставляемые production-backend’ы стрима 1 и представленные эксперименты являются нейросетевыми. Реализация ранжирует только отсечённый и квантованный целочисленный ключ $s(x, a)$, определённый в формуле (4). Скалярная оценка состояния является частным случаем $\sigma(x, a) = q(A_a(x))$, где $q : V \rightarrow \mathbb{R}$, тогда как оценщик с m выходами выдаёт все оценки действий из одного вычисления родителя. На глубине d beam search вычисляет

$$\mathcal{C}_d = \{(x, a, A_a(x)) : x \in \mathcal{F}_{d-1}, a \in [m]\},$$
$$U_d = \text{Reduce}_z(\mathcal{C}_d; \text{lexmin}(s, \text{payload})), \quad (1)$$

$$\mathcal{F}_d = \text{Top}_B(U_d; s_d^*, \tau_L), \quad (2)$$

где z — 128-битный ключ дочернего состояния, $s_d^*(z) = \min\{s(x, a) : z(A_a(x)) = z\}$, а τ_L задаёт детерминированный порядок разрешения равных оценок, индуцированный фиксированной раскладкой рангов и шардов L . Таким образом, до глобального среза дедупликация сохраняет лучшего кандидата для каждого ключа. Для насыщенного слоя, в котором к каждому удерживаемому родителю применяются все m генераторов, номинальное число кандидатов до дедупликации равно mB_{eff} . Поскольку runtime-выравнивание даёт $B_{\text{eff}} \geq B_{\text{req}}$, измеренный capacity-запрос $B_{\text{req}} = 770,883,178$ при $m = 24$ означает не менее 18,501,196,272 номинальных логических потомков на одной глубине. Материализация полных логических дочерних состояний по 120 байт потребовала бы не менее 2.220 ТБ ещё до метаданных; система избегает такого переноса на host и глобальной сортировки на одном устройстве.

Наше требование сильнее, чем GPU-ускоренное вычисление эвристики. В течение одного уровня массивы кандидатов и frontier из формулы (2) остаются в памяти GPU. CPU получает только $O(p)$ управляющих счётчиков и компактные метаданные отобранных кандидатов, но не полные массивы состояний или frontier. Термин “один согласованный поиск” имеет операционное значение: каждая GPU участвует в формуле (2), а

ранги совместно фиксируют непересекающиеся шарды одного глобально определённого следующего frontier. Это не ансамбль независимых лучей или реплик.

Вклад. Система даёт (i) GPU-резидентный hot-data контур, распределяющий один логический луч без переноса frontier через host memory; (ii) точную семантику монолитного глобального top- B по редуцированной оценке ключа при той же квантизации оценки, порядке разрешения равных оценок для фиксированной раскладки и заявленной 128-битной модели идентичности, без семантических локальных ограничений шардов; и (iii) отдельные доказательства ёмкости frontier почти в миллиард состояний на восьми GPU и измеренного высокопроизводительного исполнения на двух GPU. В совокупности это первая опубликованная высокопроизводительная multi-GPU система beam search с одним физически шардированным глобальным frontier и заявленным контрактом точности. Реализация допускает не более 128 рангов; пропускная способность более чем на двух GPU и исполнение более чем на восьми GPU пока не измерены.

2 Предшествующие работы и пробел

Параллельный beam search появился задолго до GPU: специализированные speech-ускорители исследовались уже в 1986 году [2]. Затем Wijs и Lissers распределили один глобальный луч между CPU [19], а Chong и соавторы разместили data-parallel speech beam на одной GPU [6]. Следовательно, сами по себе параллельный, распределённый и GPU beam search не новы. Современная CPU-работа удерживает лучи до 20 миллионов на общей памяти, а MPI-workers запускают независимые рандомизированные лучи [7]; Kuroiwa и Beck также исследуют глобальные и локально разделённые CPU-варианты [12].

В GPU- и эвристическом комбинаторном поиске GA* задаёт single-GPU A* без нейросетевого оценщика [21], GPUAStar удерживает neural A* на одной H200 [14], а HDA* задаёт hash-distributed CPU best-first search [11]. Ближайшим алгоритмическим предшественником является CayleyPy: его агенты для больших кубов используют одну GPU и $B = 2^{24}$ [5, 13]. AlphaCube и DeepCubeA являются соответственно однокарточным beam search и пакетным weighted A* [18, 1].

Ближайший смежный системный результат — QiankunNet-cuSCI: он генерирует конфигурации, выполняет распределённую глобальную дедупликацию, применяет нейросетевой Transformer и выбирает top- K на 64 A100 с заявленной эффективностью strong scaling 91.06% [17]. Поэтому нельзя заявлять приоритет общего конвейера expand-deduplicate-infer-select, но работа не представлена как beam search. Она удерживает $K = 32,000$ или 256,000 отобранных

SCI-конфигураций примерно из миллиарда сырых кандидатов, а её глобальные барьеры переносят полные данные кандидатов через host memory.

Другие результаты миллиардного масштаба измеряют иные объекты. GPUexplore 3.0 обходит более 3.15 миллиарда достижимых состояний на четырёх GPU, но выполняет полный обход без ограниченного top- B с ранжированием по оценке [20]. PathWeaver конвейерно обрабатывает приближённые запросы ближайших соседей по сохранённому графу [10]; multi-GPU speech- и языковые декодеры распределяют независимые высказывания, предложения или исполнение модели вокруг малых лучей, но физически не шардируют один frontier состояний [9, 3, 4].

Таблица 1 применяет единую таксономию масштаба: удерживаемая ширина луча не равна сырым кандидатам, накопленным расширениям, размеру графа или пакету независимых запросов. Предшествующие системы устанавливают важные подмножества свойств параллельного beam search, исполнения на GPU, распределённого поиска или крупномасштабного исследования состояний, но не сообщают об их полной совокупности в одной системе. Эта работа представляет первую опубликованную высокопроизводительную multi-GPU систему beam search, которая объединяет один физически шардированный логический frontier, GPU-резидентный hot-data контур кандидатов и frontier, глобальную дедупликацию по 128-битному ключу, отсутствие семантических локальных ограничений шардов и точный монолитный глобальный top- B по редуцированной оценке ключа при заявленных квантизации, порядке разрешения равных оценок и модели идентичности для запроса frontier почти в миллиард состояний. Заявление о приоритете относится только к этой полной системной совокупности.

3 Один уровень поиска в пяти GPU-стримах

На входе находится глобальный луч $\mathcal{F}_{d-1}$, распределённый между p GPU. Если GPU r владеет N_r родителями, а граф имеет m ходов, она порождает mN_r логических кандидатов. Для $y = A_a(x)$ до материализации кандидат представлен как

$$c = (z(y), s(x, a), i_{\text{parent}}, a, r_{\text{src}}), \quad (3)$$

где $z(y) \in \{0, 1\}^{128}$ — хеш дочернего состояния, $s(x, a)$ — квантованная оценка кандидата, а r_{src} указывает GPU, хранящую родителя. Само дочернее состояние восстанавливается только для проверки цели или после окончательного отбора.

Стримы 1–2: оценка и идентичность. Родители делятся на микробатчи и помещаются в переиспользуемые слоты кольцевого буфера. Стримы 1 и 2 одновременно потребляют один слот. Стрим 1 вычисляет

Таблица 1: Ближайшие предшествующие системы в единой таксономии масштаба. “Иной масштаб” намеренно не интерпретируется как удерживаемая ширина луча.

Система	Платформа и объект поиска	Удерживаемый луч	Иной масштаб	Чего нет из заявленной совокупности
Wijs–Lisser 2007	distributed CPU; один гло-	50,000	135.96M накопленных состояний	GPU-контур и device-resident scorer
Chong et al. 2008	одна 8800 GTX; активное множество	предел 20,000	модель на 1M состояний	multi-GPU frontier состояний
Frohner et al. 2022	46 CPU threads; один shared-memory луч	20,000,000	MPI: независимые лучи	GPU и распределённое исполнение одного луча
CayleyPy 2025	одна GPU; один луч графа Кэли	2^{24}	независимые агенты	multi-GPU глобальный frontier
GPUexplore 3.0	четыре V100; полное множество достижимых	не луч	$> 3.15B$ состояний	ранжированная оценка и top- B
PathWeaver 2025	четыре A6000; независимые ANNS-запросы	не сообщён	датасет 50M векторов	точная послойная семантика и большой луч
QiankunNet-cuSCI	64 A100; отобранные SCI-конфигурации	не луч; $K = 32,000/256,000$	около 1B сырых кандидатов	не заявлено как beam search; существенно меньший удерживаемый K ; перенос полных кандидатов через host
Эта работа	восемь A100; один шардированный глобальный луч	запрос 770,883,178; $\geq$ effective $\geq$ запроса	$\geq 18.501B$ номинальных сырых потомков/глубину	capacity по сводке; время и effective не записаны

выбранную функцию оценки и записывает только

$$s(x, a) = \text{rint}_{\text{RN-even}}(S \text{ clip}(\sigma(x, a), 0, q_{\max})) \in \mathbb{N}, \quad (4)$$

но не её вещественный выход. Здесь RN-even — используемое GPU-контуром округление к ближайшему с разрешением половин к чётному. Скалярный путь оценивает дочерние состояния, а путь с m выходами получает все m оценок действий для каждого родителя. Стрим 2 применяет каждый ход в регистрах, проверяет цель и записывает 128-битный хеш Зобриста. Поэтому вычисление оценки и преобразование состояний перекрываются без хранения mN_r дочерних состояний.

Стрим 3: раннее сокращение и владение. Когда группа кольцевых слотов имеет готовые оценки и хеши, стрим 3 удаляет оценки выше текущего порога T , компактизирует оставшиеся записи, сортирует их по z и объединяет равные хеши. Для каждого хеша сохраняется лексикографически минимальная пара $(s, \text{payload})$, где payload восстанавливает исходного родителя и ход. Затем уникальному кандидату назначается владелец

$$o(x) = \text{mix}(z(x)) \bmod p. \quad (5)$$

Локальные кандидаты поступают в локальный коллектор, остальные группируются по владельцу. Это первое сокращение перед коммуникацией уменьшает трафик, но не заменяет последующую глобальную дедупликацию.

Стрим 5: обмен между GPU. Стрим 5 напрямую обменивает сгруппированные по владельцу метаданные между GPU через NCCL. Правило владения даёт

$$z(x) = z(y) \implies o(x) = o(y), \quad (6)$$

поэтому дубликаты с разных исходных GPU в итоге встречаются у одного владельца. Стрим 5 выполняет только коммуникацию; полученные кандидаты поступают в тот же коллектор, что и локальные.

Стрим 4: постоянное множество выживших. Коллектор распределяет принадлежащих GPU кандидатов по логическим шардам. Каждый шард имеет два постоянных буфера: стрим 3 дописывает в один, пока стрим 4 очищает другой. Задача стрима 4 объединяет чистую и недавно дописанную области, применяет снимок T , компактизирует, сортирует по z и сохраняет лучшие метаданные для каждого хеша. Внутри шарда нет top- k и семантической квоты: шард может сохранить всех кандидатов ниже T . Поэтому шардирование не аппроксимирует срез по оценке; только порядок границы равных оценок задаётся фиксированной раскладкой.

Классический top- B и монотонное пороговое отсечение. Пусть C — полное мультимножество кандидатов, а $s(c)$ — квантованная целочисленная оценка из формулы (4). Тогда $U = \text{Reduce}_z(C; \text{lexmin}(s, \text{payload}))$ — его редукция до одной записи на ключ, которая несёт $s^*(z) = \min_{c: z(c)=z} s(c)$ и соответствующий лучший payload. Для фиксированной раскладки L ключи упорядочиваются по $\kappa_L(z) = (s^*(z), \tau_L(z))$, где меньшая оценка лучше, а τ_L — детерминированное правило реализации: сначала ранг, затем физический порядок для равных оценок. Классическая эталонная реализация материализует U , сортирует его по κ_L и при $b = \min(B, |U|)$ возвращает

$$\text{Top}_B(U) = \{z \in U : \kappa_L(z) \leq \kappa_{L,(b)}(U)\}. \quad (7)$$

Наша реализация никогда не материализует U . После обновления j пусть $C_j \subseteq C_{j+1} \subseteq C$ — уже обработанные кандидаты, а $U_j = \text{Reduce}_z(C_j)$. Область ключей U_j может только расти, а оценка существующего ключа $s_j^*(z)$ — только уменьшаться. Как только $|\text{dom } U_j| \geq B$, гистограммы текущих редуцированных оценок задают

$$\hat{T}_j = \min \left\{ t : \sum_{u=0}^t H_j(u) \geq B \right\},$$

$$T_j = \min\{T_{j-1}, \hat{T}_j\}, \quad T_0 = +\infty. \quad (8)$$

Пока $|\text{dom } U_j| < B$, полагается $\hat{T}_j = +\infty$. Стримы 3 и 4 могут удалять лишь кандидатов с $s(c) > T_j$; граница $s(c) = T_j$ сохраняется, поскольку окончательное попадание в луч на ней зависит от τ_L .

Точность. Обработка следующего кандидата либо вводит новый ключ, либо уменьшает лучшую оценку существующего, поэтому B -я наименьшая редуцированная оценка не может увеличиться; следовательно, $\hat{T}_{j+1} \leq \hat{T}_j$ и $T_{j+1} \leq T_j$. Если кандидат c удалён при обновлении j , то $s(c) > T_j$, а U_j уже содержит не менее B различных ключей с текущими лучшими оценками не выше T_j . Эти ключи останутся различными, а их финальные лучшие оценки могут только уменьшиться. Если $z(c)$ входит в их число, то c хуже уже сохранённого представителя; иначе не менее B других ключей строго лучше. Поэтому c не может определять отобранный финальный минимум. Обратно, кандидат, достигающий финального минимума ключа из глобального $\text{top-}B$, не может быть удалён: для этого уже должны существовать B различных ключей со строго меньшими оценками. Следовательно, пороговая схема сохраняет точно тот же классический $\text{top-}B$ по редуцированным оценкам ключей при той же квантизации для фиксированной L с тем же правилом разрешения равных оценок на границе.

Модель идентичности и область точности. Доказательство является точным для квантизации оценки из формулы (4), множества, индексированного 128-битным идентификатором Zobrista, и полного фиксированного порядка κ_L . Контур кандидатов не переносит все 120 логических байт состояния, поэтому равные 128-битные ключи считаются равными состояниями без полного сравнения состояний. Следовательно, отбор математически точен в явно заданной модели ключа; остаточный риск связан с коллизией 128-битного хеша, а не с аппроксимацией при шардировании, пороговом отсечении или $\text{top-}B$. Изменение числа GPU или шардов может изменить τ_L и тем самым набор состояний с равной граничной оценкой; текущая реализация не заявляет инвариантность выбранных ID к разбиению.

Финальный срез и материализация. После осушения всех пяти стримов ранги объединяют итоговые гистограммы. Если $|U| \geq B$, они выбирают минимальный T^* , для которого

$$\sum_{u=0}^{T^*} H(u) \geq B. \quad (9)$$

Все состояния ниже T^* сохраняются, а детерминированное правило обрезает его границу ровно до B состояний. Если $|U| < B$, вместо этого сохраняются все доступные ключи. Поэтому следующий frontier всегда содержит ровно $\min(B, |U|)$ состояний. Отобранные метаданные равномерно распределяются между GPU. Только после этого каждая исходная GPU восстанавливает запрошенные дочерние состояния и отправляет их непосредственно целевым GPU. CPU получает $O(p)$ векторов счётчиков и компактные метаданные отобранных кандидатов для восстановления родословной, но не полные массивы состояний или frontier.

Пять стримов управляются готовностью данных, а не выполняются как пять последовательных фаз. Стримы 1–2 почти непрерывно заполняют кольцевые слоты, пока стримы 3 и 4 сокращают предыдущие батчи, а стрим 5 обменивает удалённых выживших. Математический инвариант состоит в том, что каждый хеш, переживший финальный срез, объединён у своего единственного владельца до вычисления формулы (9). Поэтому для фиксированной раскладки распределённое выполнение реализует тот же глобальный $\text{top-}B$ по редуцированной оценке ключа, что и монолитный эталон при той же квантизации, не материализуя сырой слой кандидатов.

4 Эксперименты

Протокол. Мы оцениваем вычислительную стоимость, а не качество решения, и приводим измерение только вместе с оборудованием, нагрузкой, контрактом модели и областью. Production-время включает генерацию, дедупликацию, коммуникацию, отбор и историю; оно не взаимозаменяемо с изолированной скоростью стрима 1. Пустая ячейка означает “не измерено”, а не расчётную оценку. В таблице 2 собраны сильнейшие локально проверенные измерения.

В таблице 2 значение RTX равно полному E2E wall time, делённому на 20 завершённых глубин, а значение Kaggle — среднему для насыщенных глубин 7–10 из сводки. T4 CSV фиксирует запрошенную и выровненную эффективную ширину; скорость 30.274 М/с явно вычислена как $24 \times 82,837,504 / 65.670$ и считает логических потомков до дедупликации. Для RTX E2E-скорость не выводится, поскольку ранние глубины не насыщены. Микробенчмарки ядер и saracity-only сводка A100 не задают E2E-время глубины, поэтому помечены как неприменимые или не записанные.

Таблица 2: Измеренная производительность по платформам. E2E — критический путь production-запуска, micro — изолированный конвейер ядер, а ёмкость не содержит заявления о времени. н/в: не вычислено; н/п: неприменимо; н/з: не записано.

Платформа	Нагрузка	Луч B	Измерение	с/глубину	Сырая скорость
RTX 3070 8GB	задача 0, глубина 20	2^{22}	E2E завершён; 60.623 с	всего 3.031 сред.	н/в
RTX 3070 8GB	стрим 1, $B_\mu = 4096$, $c = 3$	—	micro завершён	н/п	48.142 М/с
Kaggle 2×T4	задача 20, радиус 4, предел 72	$B_{\text{req}} = 82,615,524$; $B_{\text{eff}} = 82,837,504$	решено; 350.402 с; CSV+сводка	65.670 сыщ.	на- 30.274 М/с
8×A100 40GB	задачи 900/950/1000, глубина 8	$B_{\text{req}} = 770,883,178$; B_{eff} не записан	стабилен; 39,745 MiB/GPU; сводка	н/з	н/з

Локальная RTX 3070. На GPU ноутбука folded-input half2 ядро повысило лучшую изолированную скорость стрима 1 примерно с 24.7 до 48.142 миллиона кандидатов/с. В Nsight время изменённого ядра сократилось с 9.678 до 3.232 с за 1,202 запуска (2.99×). Что важнее, тот же production-запуск глубины 20 сократился с 115.334 до 60.623 с, то есть ускорился в 1.90×. Старый PyTorch-запуск глубины 100 исчерпал память на глубине 33 через 329.067 с; этот незавершённый запуск не считается базой для ускорения.

Kaggle 2×T4. Два ранга совместно поддерживают один глобальный луч. Для задачи 20 запрос $B_{\text{req}} = 82,615,524$ был выровнен до $B_{\text{eff}} = 82,837,504$, а запуск с целевой окрестностью радиуса 4 решил задачу за 350.402 с при 128 шардах и $B_\mu = 4096$. При $B = 2^{26}$ увеличение числа шардов с 8 до 64 сократило среднее время насыщенных глубин 7–9 с 69.365 до 51.732 с (1.34×), сохранив точный глобальный отбор. Лучший изолированный результат стрима 1 равен 40.60 М кандидатов/с; он не используется как E2E-скорость.

Ёмкость 8×A100 и предел scale-out. Сводка exact-scale autotune фиксирует стабильное выполнение глубины 8 для трёх экземпляров Megaminx при запросе $B_{\text{req}} = 770,883,178$ с пиком 39,745 MiB на GPU. Runtime-выравнивание даёт $B_{\text{eff}} \geq B_{\text{req}}$, но точная эффективная ширина, сырые rank-логи, runtime-конфигурация, commit, хеш checkpoint и wall time не сохранились. Поэтому сводка подтверждает capacity-результат на восьми устройствах и номинальный насыщенный масштаб не менее 18.501 миллиарда логических кандидатов-потомков, но не скорость почти миллиардного запуска. Производительность измерена только до двух GPU, а ёмкость — до восьми; статические таблицы обмена сейчас допускают не более 128 рангов, но выполнение за пределами восьми GPU не измерено.

Таблица 3: Опубликованная зависимость CayleyPy от ширины луча (таблица 8 из [13]).

	W	2^{10}	2^{12}	2^{14}	2^{16}	2^{20}
Длина	—	252	202	194	190	

Таблица 4: Согласованное вычисление глубины 10 для Cube4 при $B = 4,194,304$.

GPU	Реализация	с/глубину (глубина 10)	Пар/с	Пик VRAM
T4	Pilgrim	198.153	508,007	2.90 GiB reserved
T4	Наша	181.282	555,293	1,949 MiB used
P100	Pilgrim	314.487	320,087	2.90 GiB reserved
P100	Наша	301.955	333,378	1,949 MiB used

5 Сравнение с CayleyPy и результатах на Megaminx

CayleyPy экспериментально показывает, что широкий луч повышает вероятность успеха и качество решения, но опубликованные примеры остаются однокарточными. В контролируемом эксперименте на S_{20} ширина W растёт от 2^{10} (решения нет) до 2^{20} (идеальная длина 190), а в статье о больших кубах используется $W = 2^{24}$, чтобы один агент помещался на одной GPU [13, 5]. Наша запрошенная конфигурация 770,883,178 в $735.2\times$ шире 2^{20} и в $45.95\times$ шире 2^{24} . Она также в $38.54\times$ шире крупнейшего найденного рецензируемого удерживаемого луча (20 миллионов) и в $30.84\times$ шире публичной реализации NATETRIS с 25 миллионами [8]. Эти отношения сравнивают системный масштаб, а не сложность решений на разных графах.

Дополнительно мы измерили Pilgrim и нашу реализацию поиска на одном состоянии Cube4, одном checkpoint, одной GPU, одинаковом луче и десяти слоях. Scorer — piece Transformer с 3 383 064 параметрами, 96 позициями из шести классов, $d_{\text{model}} = 256$, четырьмя слоями, восемью головами, шириной feed-forward 1024 и 24 Q-выходами; inference использует FP16 и microbatch 384. Таблица 4 показывает десятый слой, где насыщенный frontier разворачивает $24B$ пар.

5.1 Инвентаризация подходов и согласованные семейства моделей на одной T4

Таблица 6 является инвентаризацией, а не рейтингом скорости между строками. AlphaCube и DeepCubeA сохраняют свои опубликованные модели Cube3. Результаты Megaminx образуют два пересекающихся согласованных семейства: скалярное `output_dim=1` (Pilgrim Searcher, нативный CayleyPy, MultiGPUBeamSearch) и нативное `output_dim=24` (Pilgrim QSearcher, MultiGPUBeamSearch). Отношение допускается только при совпадении checkpoint, архитектуры, головы, состояния, генераторов, глубины, ширины, dtype и оборудования. CayleyPy не принимает 24-выходную голову и остаётся несовместимым без преобразований. В столбце скорости приводятся только нативно подсчитанные узлы, делённые на solver wall time; они никогда не заменяются величиной B/t .

В согласованных строках Megaminx все перечисленные реализации загружают одни и те же байты: SHA-256 скалярного checkpoint равен `7f5071e6...a539b759`, а checkpoint с 24 выходами — `e7bda332...5d2ca670`. Вторая модель не получена из скалярной. Cube4 Transformer образует отдельное согласованное сравнение и не смешивается с capacity sweep Megaminx.

Для AlphaCube с/глубину равно solver wall time, делённому на завершённую глубину поиска из возвращённого решения (20 для первых трёх строк и 17 для остальных). Для обеих реализаций Megaminx wall time делится на фиксированные восемь завершённых глубин. Эти средние включают запуск и ранние ненасыщенные слои и дополняют, а не заменяют, узлы/с.

Самая широкая скалярная точка Pilgrim, $B = 5,898,240$, завершилась за $1,462.557$ с (182.820 с/глубину) с пиком устройства $14,433$ MiB; $B = 6,291,456$ дал CUDA OOM. Для output-24 $B = 20,971,520$ завершился за 197.358 с (24.670 с/глубину) при $12,333$ MiB, тогда как $B = 23,068,672$ дал CUDA OOM. Измеренные интервалы равны соответственно $5,898,240$ – $6,291,456$ и $20,971,520$ – $23,068,672$.

Финальный MultiGPU-прогон соавтора и скалярное подтверждение v27 теперь дают измеренные пары complete/OOM для обеих голов. Скалярный запуск завершился при $29,360,128$ и дал CUDA OOM при $31,457,280$; нативная серия output-24 завершилась до $29,360,128$ и дала CUDA OOM при $33,554,432$.

Скалярный OOM повторился для каждого профиля row budget от 49,152 до 24. Общая первичная метрика равна полному wall time, делённому на восемь: 352.145 с/глубину; вспомогательное время финальной глубины равно $1,644.13$ с.

OOM output-24 повторялся при снижении runtime-профиля row budget от 2,048 до 1. Поле 1.563 с относится к последней неудачной попытке, а не к завершённой глубине.

AlphaCube материализует все $18B$ дочерних состоя-

ний Cube3 в массивах NumPy на хосте до сортировки; поэтому первый отказ является исчерпанием памяти хоста, хотя GPU reserved составляет лишь 4.685 GiB. На самых широких завершённых точках измеренная скорость стабилизируется около 40 – 44 тысяч узлов/с. DeepCubeA не имеет постоянного луча: это weighted A*, а его нативным масштабом служат search/NN batch $10,000/10,000$, поэтому приписывать ему максимальный фронт было бы некорректно. Завершённый T4-запуск выполнил 55 поисковых итераций за 146.474 с, то есть 2.663 с на одну нативную A*-итерацию; это не измерение beam-глубины. За запуск обработано $6,134,352$ узла со скоростью $41,880$ узлов/с и получено решение длины 21.

Для согласованной скалярной пары Megaminx CayleyPy commit `5b2b53f` использует iterated search с `history_depth=0`. Широкий протокол завершил каждую степень двойки от 2^{10} до 2^{24} и получил CUDA OOM на 2^{25} ; оценка по тренду памяти совпала с 2^{24} , поэтому лишняя подтверждающая точка не запускалась. Более раннее целевое измерение с тем же контрактом сохраняет более узкую пару $20,971,520$ complete против $21,037,056$ CUDA OOM, но не выдаётся за часть широкой сетки. Внутренний путь расширения обошёл публичный hook счётчика, поэтому валидны только wall time и память. MultiGPU также сохранил end-to-end wall, но не валидный счётчик узлов. Финальный скалярный прогон измерил $29,360,128$ complete при $14,215$ MiB против $31,457,280$ CUDA OOM при $14,909$ MiB. Согласованный прогон output-24 измерил $29,360,128$ complete против $33,554,432$ CUDA OOM. Отказы устаревшего статического профиля и отменённые v10/v11 исключены. В последней согласованной точке широкой сетки CayleyPy занял $4,741.581$ с, а исторический MultiGPU-запуск $1,728.460$ с при $B = 16,777,216$ (wall ниже в 2.74 раза). На более ранней целевой точке $B = 20,971,520$ wall MultiGPU ниже в 1.95 раза; это отношения согласованного end-to-end wall, а не восстановленные отношения nodes/s.

Эти скалярные отношения используют один идентичный контракт скалярной модели и не сравниваются с более быстрой строкой Pilgrim QSearcher. 24-выходная модель оценивает все 24 действия для каждого родителя и имеет другую архитектуру; поэтому разница scalar-Q является разницей контракта модели, а не ускорением алгоритма. Отдельное сравнение Cube4 использует идентичный 24-выходный Q-checkpoint для Pilgrim и MultiGPUBeamSearch. Значение output-24 для CayleyPy не выводится.

Таким образом, в согласованной точке с Pilgrim наша пропускная способность на десятом слое выше в $1.093\times$ на T4 и в $1.042\times$ на P100. Резерв аллокатора PyTorch и полное использование устройства по `nvidia-smi` являются разными метриками памяти, поэтому таблица не утверждает отношение расхода памяти; она устанавливает, что обе реализации завершают луч 4M.

Для Megaminx-superflip найденная публичная хроно-

Таблица 5: Точные архитектуры нейронных сетей в приведённых тестах. Число параметров относится к обучаемой исходной модели до свёртки batch normalization.

Семейство модели	Параметры	Где используется	Последовательность блоков
Cube3 AlphaCube 0.1.6 large	118,939,666	AlphaCube	one-hot 324 → 8×[Linear 4096, ReLU, BN] → Linear 18
Cube3 DeepCubeA official cube3	14,663,001	DeepCubeA	one-hot 324 → Linear 5000, BN, ReLU → Linear 1000, BN, ReLU → 4×[Linear 1000, BN, ReLU, Linear 1000, BN, сложение residual, ReLU] → Linear 1
Megaminx scalar MLP	34,766,849	Pilgrim Searcher; CayleyPy; MultiGPU	позиция-класс 120 × 120 → эквивалент EmbeddingBag Linear 2048, BN, ReLU → Linear 512, BN, ReLU → 8×[Linear 512, BN, ReLU, Linear 512, BN, сложение residual, ReLU] → Linear 1
Megaminx QMLP2RB	23,978,008	Pilgrim QSearcher; MultiGPU	позиция-класс 120 × 120 → эквивалент EmbeddingBag Linear 1536, BN, ReLU → Linear 512, BN, ReLU → 2×[Linear 512, BN, ReLU, Linear 512, BN, сложение residual, ReLU] → Linear 24
Cube4 Q Transformer	3,383,064	только согласованный Cube4	96 токенов / 6 классов → проекция + CLS → 4×[pre-LN, 8-головое attention ($d = 256$), residual, pre-LN, FFN 256–1024–256 ReLU, residual] → LN → Linear 24

Таблица 6: Инвентаризация нативных измерений на одной видимой T4. Строки с разными контрактами моделей не сравниваются; максимум измерен.

Подход	Нагрузка / нативный масштаб	Макс. complete	Время (с)	с/нативный шаг	Узлы/с	Граница/статус
AlphaCube 0.1.6	Cube3 large , beam	1,572,864	461.488	27.146/глубину	42,387	1,703,936 host SIGKILL
DeepCubeA	Cube3 A*, 55 итераций	н/п	146.474	2.663/итерацию	41,880	complete, длина 21
Pilgrim Searcher	Megaminx scalar beam, глубина 8	5,898,240	1,462.557	182.820/глубину	не измерено	6,291,456 OOM
Pilgrim QSearcher	Megaminx output-24 beam, глубина 8	20,971,520	197.358	24.670/глубину	не измерено	23,068,672 OOM
нативный CayleyPy	Megaminx scalar beam, глубина 8	20,971,520	4,258.602	532.325/глубину	не измерено	broad: 2 ²⁴ complete, 2 ²⁵ OOM
MultiGPUBeamSearch	Megaminx scalar beam, глубина 8	29,360,128	2,817.158	352.145/глубину	не измерено	31,457,280 OOM
MultiGPUBeamSearch	Megaminx output-24 beam, глубина 8	29,360,128	116.904	14.613/глубину	не измерено	33,554,432 OOM

Таблица 7: Завершённый sweep AlphaCube на 1xT4.

B	Время (с)	с/глубину	Узлы	Узлы/с
1,024	1.034	0.052	18,720	18,108
4,096	1.199	0.060	73,484	61,309
16,384	5.014	0.251	282,380	56,321
65,536	18.543	1.091	903,448	48,722
131,072	37.818	2.224	1,755,416	46,418
262,144	79.166	4.657	3,459,352	43,698
524,288	170.671	10.039	6,867,224	40,237
786,432	228.938	13.467	10,124,136	44,222
1,048,576	303.870	17.875	13,269,864	43,670
1,310,720	382.158	22.480	16,415,592	42,955
1,572,864	461.488	27.146	19,561,320	42,387

логия улучшается от 55 через 54 и 53 до 51 хода; следовательно, непосредственно предшествовавшее публичное значение было 53, а не 55 [16, 15]. Путь из 51 шага использует ту же метрику 24 знаковых поворотов одной грани на 72 градуса и имеет публичный wall time 22,600.2 с. Независимый CPU replay сохранённого submission достигает точного центрального состояния с нулём различий; SHA-256 пути и файла записаны в сопровождающем аудите доказательств. В цитируемой публичной хронологии нет более короткого пути в той же метрике, но формального реестра и доказательства оптимальности нет. Поэтому мы называем его кратчайшим найденным публичным решением в той же метрике, а не безусловным мировым рекордом. Отсутствующий манифест этого запуска отделён от saracity-сводки глубины 8 для запроса 770,883,178 состояний.

6 Код и артефакты

Исходники статьи, индекс доказательств и машиночитаемые таблицы общедоступны по адресу <https://github.com/TryDotAtwo/MultiGPUBeamSearchPaper>. Релиз v1.0.0 содержит обе PDF-версии и согласованные checkpoints: <https://github.com/TryDotAtwo/MultiGPUBeamSearchPaper/releases/tag/v1.0.0>.

Исходники solver и его готовый к запуску на GPU Megaminx-релиз общедоступны по адресам <https://github.com/TryDotAtwo/MultiGPUBeamSearch> и <https://github.com/TryDotAtwo/MultiGPUBeamSearch/releases/tag/megaminx-native-cluster-latest>.

Публичный самодостаточный одно-T4 датасет для воспроизведения доступен по адресу <https://www.kaggle.com/datasets/trydotatwo/gpu-beam-search-paper-1xt4-reviewer-notebooks>.

Четыре импортируемых исходника ноутбуков для широких sweep и подтверждений границ Pilgrim и MultiGPUBeamSearch, а также runners CayleyPy, AlphaCube и DeepCubeA, версионированы в репозитории статьи. Они используют публичный датасет через обычный workflow Kaggle Import Notebook и Save Version и не зависят от видимости сохранённой страницы kernel владельца. Запись для восьми A100 даёт доказательства ёмкости и памяти, но не wall time или точную эффективную ширину; запись для двух T4 отдельно даёт измеренную пропускную способность. Отсутствующая метрика времени, ёмкости или масштабирования не выводится из другого запуска.

Таблица 8: Завершённый sweep нативного CayleyPy на 1xT4 по степеням двойки. Память — пик CUDA-аллокатора PyTorch; последняя строка — первый отказ широкой сетки.

B	Время (с)	с/глубину	Allocated (MiB)	Reserved (MiB)	Статус
1,024	0.930	0.116	85.9	92.0	complete
2,048	1.237	0.155	96.3	112.0	complete
4,096	2.189	0.274	117.3	128.0	complete
8,192	3.828	0.478	158.6	230.0	complete
16,384	6.776	0.847	241.5	292.0	complete
32,768	13.722	1.715	407.5	510.0	complete
65,536	31.399	3.925	739.3	858.0	complete
131,072	63.664	7.958	772.2	1,180.0	complete
262,144	114.551	14.319	834.8	1,168.0	complete
524,288	219.991	27.499	962.1	2,156.0	complete
1,048,576	430.789	53.849	1,216.7	3,234.0	complete
2,097,152	745.341	93.168	1,726.0	2,740.0	complete
4,194,304	1,320.000	165.000	2,745.2	4,492.0	complete
8,388,608	2,489.356	311.169	5,155.3	6,462.0	complete
16,777,216	4,741.581	592.698	10,233.7	14,652.0	complete
33,554,432	—	—	12,922.2	14,484.0	CUDA OOM

Таблица 9: Финальные скалярные граничные точки MultiGPU на 1xT4.

B	Время (с)	с/глубину	Пик устройства (MiB)	Статус
25,165,824	3,027.673	378.459	12,267	complete
27,262,976	3,080.206	385.026	13,241	complete
29,360,128	2,817.158	352.145	14,215	complete
31,457,280	—	—	14,909	CUDA OOM

Таблица 10: Финальный нативный output-24 sweep MultiGPU на 1xT4.

B	Время (с)	с/глубину	Пик устройства (MiB)	Статус
1,048,576	8.296	1.037	773	complete
4,194,304	22.735	2.842	2,253	complete
8,388,608	41.091	5.136	4,159	complete
16,777,216	77.460	9.683	7,983	complete
20,971,520	96.124	12.015	9,893	complete
25,165,824	111.130	13.891	11,799	complete
29,360,128	116.904	14.613	13,717	complete
33,554,432	1.563	—	14,909	CUDA OOM

7 Заключение

Эта работа устанавливает один GPU-резидентный распределённый beam search, а не набор локальных суррогатов. Для фиксированной раскладки его потоковый контур expand-deduplicate-select, без семантических локальных ограничений шардов, математически эквивалентен монолитному глобальному top- B по редуцированной оценке ключа при той же квантизации, порядке разрешения равных оценок и заявленной 128-битной модели идентичности. Результат на восьми A100 устанавливает запрос frontier из 770,883,178 удерживаемых состояний и не менее 18.501 миллиарда номинальных потомков до дедупликации на насыщенную глубину; он не содержит wall time или точную эффективную ширину. Отдельный результат на двух T4 измеряет end-to-end скорость 30.274 миллиона логических потомков/с, поэтому высокопроизводительное исполне-

Таблица 11: Согласованный скалярный Megaminx-sweep на 1xT4. Ни одна реализация не предоставляет валидный счётчик узлов для этих точек.

MultiGPU			CayleyPy	
B	Время (с)	с/глубину	Время (с)	с/глубину
65,536	12.537	1.567	31.399	3.925
262,144	39.935	4.992	114.551	14.319
1,048,576	152.604	19.076	430.789	53.849
4,194,304	476.007	59.501	1,320.000	165.000
8,388,608	892.948	111.619	2,489.356	311.169
16,777,216	1,728.460	216.058	4,741.581	592.698
20,971,520	2,179.990	272.499	4,258.602	532.325
25,165,824	2,416.150	302.019	—	—

ние продемонстрировано до двух GPU, а более сильное масштабирование остаётся неизмеренным. Проект также предоставляет replay-valid путь Megaminx-superflip длиной 51 ход — кратчайшее публичное решение в той же метрике, установленное сопровождающими доказательствами. Это первая опубликованная высокопроизводительная multi-GPU система beam search с одним физически шардированным глобальным frontier и заявленным контрактом точности при масштабе удерживаемого frontier почти в миллиард состояний.

Список литературы

- [1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi. Solving the rubik's cube with deep reinforcement learning and search. *Nature Machine Intelligence*, 1:356–363, 2019.
- [2] T. S. Anantharaman and R. Bisiani. A hardware accelerator for speech recognition algorithms. In *Proceedings of the 13th Annual International Symposium on Computer Architecture*, pages 216–223, 1986.
- [3] H. Braun, J. Luitjens, R. Leary, T. Kaldewey, and D. Povey. GPU-accelerated viterbi exact lattice decoder for batched online and offline speech recognition. In *2020*

IEEE International Conference on Acoustics, Speech and Signal Processing, 2020.

- [4] W. Chan et al. Efficient beam search for large language models using trie-based decoding. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14795–14807, 2025.
- [5] A. Chervov et al. A machine learning approach that beats large rubik’s cubes. *arXiv preprint arXiv:2502.13266*, 2025.
- [6] J. Chong, Y. Yi, A. Faria, N. R. Satish, and K. Keutzer. Data-parallel large vocabulary continuous speech recognition on graphics processors. Technical Report UCB/EECS-2008-69, University of California, Berkeley, 2008.
- [7] N. Frohner, J. Gmys, N. Melab, G. R. Raidl, and E.-G. Talbi. Parallel beam search for combinatorial optimization. In *Proceedings of the 51st International Conference on Parallel Processing Workshops*, 2022.
- [8] HATETRIS public beam-search contributors. HATETRIS public beam-search implementation, 2026. Public repository, accessed 25 August 2026.
- [9] M. Junczys-Dowmunt, T. Dwojak, and H. Hoang. Is neural machine translation ready for deployment? a case study on 30 translation directions. In *Proceedings of the 13th International Conference on Spoken Language Translation*, 2016.
- [10] S. Kim, S. Park, S. U. Noh, J. Hong, T. Kwon, H. Lim, and J. Lee. PathWeaver: A high-throughput multi-gpu system for graph-based approximate nearest neighbor search. In *2025 USENIX Annual Technical Conference*, pages 1501–1517, 2025.
- [11] A. Kishimoto, A. Fukunaga, and A. Botea. Evaluation of a simple, scalable, parallel best-first search strategy. *Artificial Intelligence*, 195:222–248, 2013.
- [12] R. Kuroiwa and J. C. Beck. Parallel beam search algorithms for domain-independent dynamic programming. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 20743–20750, 2024.
- [13] A. Soibelman et al. CayleyPy RL: Pathfinding and reinforcement learning on cayley graphs. *arXiv preprint arXiv:2502.18663*, 2025.
- [14] M. Soltani and F. Agostinelli. GPU-accelerated A* search with deep neural network heuristics. In *Proceedings of the Nineteenth International Symposium on Combinatorial Search*, 2026.
- [15] SpeedSolving forum contributor. Development of a megaminx solver: 51-move superflip report. SpeedSolving forum post 1705451, 2026.
- [16] SpeedSolving forum contributor. Development of a megaminx solver: 53-move superflip report. SpeedSolving forum post 1700151, 2026.
- [17] D. Sun, B. Kan, H. Long, H. Zhao, H. Li, Y. Liu, P. Zhou, A. Feng, W. Huang, Y. Gu, Z. Li, H. Shang, Y. Zhang, D. Tao, N. Sun, and G. Tan. A fully GPU-accelerated framework for high-performance configuration interaction selection with neural network quantum states. In *Proceedings of the 35th International Symposium on High-Performance Parallel and Distributed Computing*, 2026.
- [18] K. Takano. Self-supervision is all you need for solving rubik’s cube. *Transactions on Machine Learning Research*, 2023.
- [19] A. Wijs and B. Lisser. Distributed extended beam search for quantitative model checking. In *Model Checking Software: 14th International SPIN Workshop*, pages 166–184, 2007.
- [20] A. Wijs and M. Osama. The fast and the capacious: Memory-efficient multi-GPU accelerated explicit state space exploration with GPUexplore 3.0. *Frontiers in High Performance Computing*, 2, 2024.
- [21] Y. Zhou and J. Zeng. Massively parallel A* search on a GPU. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.